FlowSense

Quantitative engine methodology

Milestone 02 | Implementation note | 3 November 2026

Abstract

The FlowSense engine turns daily investor flow observations into descriptive measures of unusual activity, accumulated flow, positioning changes, and cross-asset differences. It uses transparent calculations with explicit history requirements. This note documents the implemented Python engine, explains its choices, and distinguishes numerical correctness from economic or predictive validity.

The current inputs are fabricated synthetic data. No BNY iFlow data or proprietary analytics were used. The engine does not estimate investor holdings, forecast returns, infer market regimes, or recommend trades. No LLM or dashboard is part of this milestone.

1. Input contract and processing

Input columns are date, asset_class, region, net_flow, and positioning_proxy. The current dataset has 523 Monday-Friday dates over 2024-2025 and nine asset-region series: equities, government bonds, and FX in the US, Europe, and Asia. Daily net flows are expressed in synthetic USD-equivalent millions; positioning is a unitless index in [-100, 100].

The engine copies its inputs, parses and sorts dates, and computes each metric separately within each asset-region group. It accepts a subset of the available groups, but every included group must cover the same complete weekday calendar. A missing observation is not interpreted as a zero flow.

Validation rejects empty inputs, missing required values or columns, duplicate date/asset/region keys, weekend dates, dates with time components, unknown assets or regions, nonfinite numeric inputs, and positioning outside its allowed bounds. This weekday convention matches the generator; a production feed would need appropriate holiday and publication calendars.

2. Defaults and notation

Setting	Default	Role
window (W)	20 observations	Prior-history z-score baseline and current rolling flow sum
position_lag (H)	5 observations	Horizon for positioning change

Let F[t] be daily net flow and Q[t] the supplied positioning proxy for one group. The index t counts business-day observations, not calendar days. Twenty and five are convenient month-like and week-like horizons, not empirically optimized values. The implementation requires integer W >= 2 and integer H >= 1.

3. Rolling flow z-score

The baseline uses the previous W observations and excludes today. The mean and sample standard deviation are:

```
m[t] = (1/W) * sum(F[t-j], j = 1,...,W)
s[t] = sqrt(sum((F[t-j] - m[t])^2, j = 1,...,W) / (W-1))
z[t] = (F[t] - m[t]) / s[t]
```

Implementation: flow.shift(1).rolling(W, min_periods=W), followed by mean() and std(ddof=1). Outputs are prior_flow_mean, prior_flow_std, and flow_zscore. The sample denominator W-1 is a familiar descriptive convention; with autocorrelated observations it is not a guarantee of an unbiased estimate of an underlying process variance.

Why exclude today? The current observation should not change the baseline used to assess its own unusualness. This also ensures the baseline is available before today's flow is known. The z-score itself requires today's flow and is therefore an end-of-observation measure, not a forecast or a tradable signal available before publication.

A positive z-score means today's flow is above its recent mean; it does not necessarily mean an inflow. For example, a flow of -20 against a mean of -50 and standard deviation of 10 has z = +3, despite being an outflow. Negative scores similarly indicate below-baseline activity rather than necessarily negative flow.

Worked example

Quantity	Value for a three-day illustration
Prior daily flows	1, 2, 3
Prior mean and sample standard deviation	Mean = 2; standard deviation = 1
Today's flow	4
Today's z-score	(4 - 2) / 1 = 2

Availability and statistical interpretation

The first W dates have no z-score: the first available score is on observation W+1. If the prior standard deviation is exactly zero, the score remains unavailable even if today's flow differs from that baseline. The engine uses NaN instead of inventing zero or infinity. Very small nonzero standard deviations can still produce large scores; no floor or clipping is currently applied.

A z-score of 2 means two baseline sample standard deviations above the baseline mean. It is not automatically a p-value, a 95% confidence claim, or evidence of a profitable anomaly. Serial dependence, changing volatility, short windows, and regime shifts affect its interpretation. The rolling baseline also adapts to a sustained episode, so persistent inflows need not retain large z-scores.

4. Cumulative and rolling net flows

```
C[t] = sum(F[j], j = first input observation,...,t)
S[t] = sum(F[t-j], j = 0,...,W-1)
```

cumulative_flow is the running total from the start of the provided input. It retains all observations and is recomputed from net_flow rather than trusting the stored generator total. Restricting the input history resets its starting point; totals from different start dates are not directly comparable.

rolling_net_flow sums W observations ending today. Unlike the z-score baseline, it includes today because it describes the most recent W-day accumulated flow. Its first W-1 values are unavailable. Both measures retain the input flow units, synthetic USD-equivalent millions. Neither measures portfolio size or returns.

5. Positioning changes

```
position_change_1d[t] = Q[t] - Q[t-1]
position_change[t] = Q[t] - Q[t-H]
```

The engine reads the positioning proxy supplied by the generator; it does not create a new estimate of holdings. It computes changes as index-point differences, not percentage changes. A move from -10 to +5 is +15 index points. The first daily change is unavailable; an H-observation change needs H earlier observations, making its first H rows unavailable.

The synthetic proxy smooths normalized flow and maps it through 100 * tanh(value). Consequently, a change is a summary of the same underlying flows, not independent positioning evidence. Saturation near -100 or +100 compresses changes. A small index change near a bound should not be interpreted as small actual investor activity.

6. Cross-asset divergences

Within each region, the engine compares three ordered pairs: equities minus government bonds; equities minus FX; government bonds minus FX. It joins dates one-to-one and compares each asset against its own recent flow baseline.

```
zscore_spread[A,B,t] = z[A,t] - z[B,t]
opposite_flow_direction = (F[A,t] > 0 and F[B,t] < 0)
or (F[A,t] < 0 and F[B,t] > 0)
```

A positive spread means A's flow is stronger relative to its own baseline than B's. It does not establish larger raw flows or capital rotation between the assets. For z[A] = 2 and z[B] = -1, the spread is 3. Both raw flows could still be positive; the separate Boolean flag records whether their actual signs are strictly opposite. Zero is not an opposite sign.

The spread is not itself a standardized z-score: its variance depends on both inputs and their dependence. No correlation adjustment, significance test, threshold, automatic risk-on/off classification, or trade rule is implemented. If either score is unavailable, the spread is unavailable. With fewer than two assets in a region, no pair rows are returned.

7. Outputs, reproducibility, and execution

The source SQLite database is opened read-only. The command writes two CSVs, replacing prior files in the selected output directory, and prints the latest per-series z-score and positioning change. Calculations are deterministic given the same inputs and settings; the analytics engine draws no random numbers.

Output	Contents
data/analytics/flow_metrics.csv	Input fields; prior mean/std; flow z-score; cumulative and rolling flows; daily and H-day proxy changes; window and position_lag
data/analytics/cross_asset_divergences.csv	Date, region, asset_a, asset_b, zscore_a, zscore_b, signed zscore_spread, opposite_flow_direction

Default full-dataset runs produce 4,707 metric rows and 4,707 pair rows. Early rows are retained with unavailable metrics, exported as empty CSV cells. The defaults yield 180 unavailable individual z-scores (20 x 9) and 45 unavailable five-day positioning changes (5 x 9). Pair files should be interpreted alongside the settings used for their corresponding metrics file.

```
source .venv/bin/activate
python -m scripts.analyze_data
python -m unittest discover -s tests -v
```

CLI options are --database, --output-dir, --window, and --position-lag. The reusable API is load_flows(path), calculate_metrics(data, window=20, position_lag=5), followed by calculate_divergences(metrics). The last function expects the validated output of calculate_metrics.

8. Validation and scope of evidence

Seven analytics tests cover hand-calculated examples; history requirements and zero variance; current-observation exclusion and future-data invariance; independent group calculations and input preservation; pair orientation, spreads, and zero-flow handling; invalid inputs; and the SQLite-to-analytics path, including source-file preservation. Together with four generator tests, all 11 tests pass. This verifies implementation behavior, not investment usefulness.

9. Assumptions and limitations

The engine assumes daily observations are comparable within a series and that input flow definitions and proxy units are consistent over time. It uses no market-size or assets-under-custody normalization, robust outlier estimator, seasonal adjustment, multiple-testing correction, uncertainty interval, or transaction-cost model. Z-scoring helps compare relative deviations but does not make asset distributions economically equivalent.

Simulator regime and volatility labels are deliberately excluded from input calculations. Only observations through the current date are used, but real deployment would also need publication timestamps, revision histories, and timing controls. Synthetic test results cannot establish real-data robustness or applicability to BNY iFlow. Parameters should be reviewed through sensitivity analysis and independent out-of-sample evaluation before empirical claims.

Implementation sources: flowsense/analytics.py, scripts/analyze_data.py, and tests/test_analytics.py, inspected 3 November 2026. This note documents implemented descriptive formulas; it does not claim proprietary or industry-standard calibration.